JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Class AsynchronousChannelGroup

java.lang.Object
    java.nio.channels.AsynchronousChannelGroup

```
public abstract class AsynchronousChannelGroup
extends Object
```

A grouping of asynchronous channels for the purpose of resource sharing.

An asynchronous channel group encapsulates the mechanics required to handle the completion of I/O operations initiated by `asynchronous channels` that are bound to the group. A group has an associated thread pool to which tasks are submitted to handle I/O events and dispatch to `completion-handlers` that consume the result of asynchronous operations performed on channels in the group. In addition to handling I/O events, the pooled threads may also execute other tasks required to support the execution of asynchronous I/O operations.

An asynchronous channel group is created by invoking the `withFixedThreadPool` or `withCachedThreadPool` methods defined here. Channels are bound to a group by specifying the group when constructing the channel. The associated thread pool is *owned* by the group; termination of the group results in the shutdown of the associated thread pool.

In addition to groups created explicitly, the Java virtual machine maintains a system-wide *default group* that is constructed automatically. Asynchronous channels that do not specify a group at construction time are bound to the default group. The default group has an associated thread pool that creates new threads as needed. The default group may be configured by means of system properties defined in the table below. Where the `ThreadFactory` for the default group is not configured then the pooled threads of the default group are `daemon` threads.

System property	Description
`java.nio.channels.DefaultThreadPool.threadFactory`	

	The value of this property is taken to be the fully-qualified name of a concrete `ThreadFactory` class. The class is loaded using the system class loader and instantiated. The factory's `newThread` method is invoked to create each thread for the default group's thread pool. If the process to load and instantiate the value of the property fails then an unspecified error is thrown during the construction of the default group.
`java.nio.channels.DefaultThreadPool.initialSize`	The value of the `initialSize` parameter for the default group (see `withCachedThreadPool`). The value of the property is taken to be the `String` representation of an `Integer` that is the initial size parameter. If the value cannot be parsed as an `Integer` it causes an unspecified error to be thrown during the construction of the default group.

Threading

The completion handler for an I/O operation initiated on a channel bound to a group is guaranteed to be invoked by one of the pooled threads in the group. This ensures that the completion handler is run by a thread with the expected *identity*.

Where an I/O operation completes immediately, and the initiating thread is one of the pooled threads in the group then the completion handler may be invoked directly by the initiating thread. To avoid stack overflow, an implementation may impose a limit as to the number of activations on the thread stack. Some I/O

operations may prohibit invoking the completion handler directly by the initiating thread (see `accept`).

Shutdown and Termination

The `shutdown` method is used to initiate an *orderly shutdown* of a group. An orderly shutdown marks the group as shutdown; further attempts to construct a channel that binds to the group will throw `ShutdownChannelGroupException`. Whether or not a group is shutdown can be tested using the `isShutdown` method. Once shutdown, the group *terminates* when all asynchronous channels that are bound to the group are closed, all actively executing completion handlers have run to completion, and resources used by the group are released. No attempt is made to stop or interrupt threads that are executing completion handlers. The `isTerminated` method is used to test if the group has terminated, and the `awaitTermination` method can be used to block until the group has terminated.

The `shutdownNow` method can be used to initiate a *forceful shutdown* of the group. In addition to the actions performed by an orderly shutdown, the `shutdownNow` method closes all open channels in the group as if by invoking the `close` method.

Since:

1.7

See Also:

`AsynchronousSocketChannel.open(AsynchronousChannelGroup)`, `AsynchronousServerSocketChannel.open(AsynchronousChannelGroup)`

Constructor Summary

Constructors

Modifier	Constructor	Descri
protected	**`AsynchronousChannelGroup`**`(`**`AsynchronousChannelProvider`** `provider)`	Initiali a new instan of this class.

Method Summary

Concrete

Modifier and Type	Method	D
abstract boolean	**`awaitTermination`**`(long timeout,` **`TimeUnit`** `unit)`	

abstract boolean	**isShutdown**()
abstract boolean	**isTerminated**()
AsynchronousChannelProvider	**provider**()
abstract void	**shutdown**()
abstract void	**shutdownNow**()
static **AsynchronousChannelGroup**	**withCachedThreadPool**(**ExecutorService** executor, int initialSize)
static **AsynchronousChannelGroup**	**withFixedThreadPool**(int nThreads, **ThreadFactory** threadFactory)
static **AsynchronousChannelGroup**	**withThreadPool**(**ExecutorService** executor)

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AsynchronousChannelGroup

```
protected AsynchronousChannelGroup(AsynchronousChannelProvider provider)
```

Initialize a new instance of this class.

Parameters:

provider - The asynchronous channel provider for this group

Method Detail

provider

```
public final AsynchronousChannelProvider provider()
```

Returns the provider that created this channel group.

Returns:

The provider that created this channel group

withFixedThreadPool

```
public static AsynchronousChannelGroup withFixedThreadPool(int nThreads,
                                                           ThreadFactory threadFa
                                                    throws IOException
```

Creates an asynchronous channel group with a fixed thread pool.

The resulting asynchronous channel group reuses a fixed number of threads. At any point, at most nThreads threads will be active processing tasks that

are submitted to handle I/O events and dispatch completion results for operations initiated on asynchronous channels in the group.

The group is created by invoking the `openAsynchronousChannelGroup(int,ThreadFactory)` method of the system-wide default `AsynchronousChannelProvider` object.

Parameters:

`nThreads - The number of threads in the pool`

`threadFactory - The factory to use when creating new threads`

Returns:

`A new asynchronous channel group`

Throws:

`IllegalArgumentException - If nThreads <= 0`

`IOException - If an I/O error occurs`

withCachedThreadPool

```
public static AsynchronousChannelGroup withCachedThreadPool(ExecutorService execu
                                                             int initialSize)
                                                      throws IOException
```

Creates an asynchronous channel group with a given thread pool that creates new threads as needed.

The `executor` parameter is an `ExecutorService` that creates new threads as needed to execute tasks that are submitted to handle I/O events and dispatch completion results for operations initiated on asynchronous channels in the group. It may reuse previously constructed threads when they are available.

The `initialSize` parameter may be used by the implementation as a *hint* as to the initial number of tasks it may submit. For example, it may be used to indicate the initial number of threads that wait on I/O events.

The executor is intended to be used exclusively by the resulting asynchronous channel group. Termination of the group results in the orderly `shutdown` of the executor service. Shutting down the executor service by other means results in unspecified behavior.

The group is created by invoking the `openAsynchronousChannelGroup(ExecutorService,int)` method of the system-wide default `AsynchronousChannelProvider` object.

Parameters:

`executor - The thread pool for the resulting group`

`initialSize - A value >=0 or a negative value for implementation specific default`

Returns:

A new asynchronous channel group

Throws:

IOException - If an I/O error occurs

See Also:

Executors.newCachedThreadPool()

withThreadPool

```
public static AsynchronousChannelGroup withThreadPool(ExecutorService executor)
                                                throws IOException
```

Creates an asynchronous channel group with a given thread pool.

The executor parameter is an ExecutorService that executes tasks submitted to dispatch completion results for operations initiated on asynchronous channels in the group.

Care should be taken when configuring the executor service. It should support *direct handoff* or *unbounded queuing* of submitted tasks, and the thread that invokes the execute method should never invoke the task directly. An implementation may mandate additional constraints.

The executor is intended to be used exclusively by the resulting asynchronous channel group. Termination of the group results in the orderly shutdown of the executor service. Shutting down the executor service by other means results in unspecified behavior.

The group is created by invoking the openAsynchronousChannelGroup(ExecutorService,int) method of the system-wide default AsynchronousChannelProvider object with an initialSize of 0.

Parameters:

executor - The thread pool for the resulting group

Returns:

A new asynchronous channel group

Throws:

IOException - If an I/O error occurs

isShutdown

```
public abstract boolean isShutdown()
```

Tells whether or not this asynchronous channel group is shutdown.

Returns:

`true if this asynchronous channel group is shutdown or has been marked for shutdown.`

isTerminated

```
public abstract boolean isTerminated()
```

Tells whether or not this group has terminated.

Where this method returns `true`, then the associated thread pool has also `terminated`.

Returns:

`true if this group has terminated`

shutdown

```
public abstract void shutdown()
```

Initiates an orderly shutdown of the group.

This method marks the group as shutdown. Further attempts to construct channel that binds to this group will throw `ShutdownChannelGroupException`. The group terminates when all asynchronous channels in the group are closed, all actively executing completion handlers have run to completion, and all resources have been released. This method has no effect if the group is already shutdown.

shutdownNow

```
public abstract void shutdownNow()
                            throws IOException
```

Shuts down the group and closes all open channels in the group.

In addition to the actions performed by the `shutdown` method, this method invokes the `close` method on all open channels in the group. This method does not attempt to stop or interrupt threads that are executing completion handlers. The group terminates when all actively executing completion handlers have run to completion and all resources have been released. This method may be invoked at any time. If some other thread has already invoked it, then another invocation will block until the first invocation is complete, after which it will return without effect.

Throws:

IOException - If an I/O error occurs

awaitTermination

```
public abstract boolean awaitTermination(long timeout,
                                         TimeUnit unit)
                                  throws InterruptedException
```

Awaits termination of the group.

This method blocks until the group has terminated, or the timeout occurs, or the current thread is interrupted, whichever happens first.

Parameters:

timeout - The maximum time to wait, or zero or less to not wait

unit - The time unit of the timeout argument

Returns:

true if the group has terminated; false if the timeout elapsed before termination

Throws:

InterruptedException - If interrupted while waiting

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.